Exact PTO Equivalence Binary Multiplication

Annealed Controller

`analyses/annealing/DFST_Softmax/mult2/hidden_64_data_100/`

1 Statement and Scope

Controllers. The exported table `minimized_transition_table.pdf` defines controller C, with initial state 0 and halt state 51. The function `mult_grid(2)` defines expert $\mathcal{E}$, with initial state 0 and halt state 31. The adjacent expert export matches this function entry for entry. All tables retain the exported state numbers.

Input Domain. Let $D = \{0, 1\}$. Row 0 initially contains the contiguous word $a\,x\,b$, starting at column 0, with $a \in D^m$, $b \in D^n$, and $m, n \geq 1$. All remaining cells contain blanks. The head starts at $(0, 0)$. Write $A = \mathrm{val}_2(a)$ and $B = \mathrm{val}_2(b)$. The domain permits arbitrary operand lengths, unequal lengths, and leading zeros.

Execution Convention. The grid uses $\mathbb{Z}^2$, with coordinates increasing rightward and downward. The symbol λ denotes a blank. A transition (q', w, M) writes w, performs motion M, and enters state q'. Motions L, R, D, U move one cell. Motion S writes and stops without moving.

Policy-Trajectory Observations. For input $a\,x\,b$, define the policy-trajectory observation (PTO) by

$$\mathrm{PTO}(a\,x\,b) = \big((r_t, w_t, M_t)\big)_{t=0}^{T-1},$$

where r_t, w_t, and M_t denote the read symbol, written symbol, and motion at controller step t. A finite execution contributes the final S action exactly once, without halt-loop padding.

Theorem 1 (Exact PTO Equivalence)**.** *For every input in the stated domain,* $\mathrm{PTO}_C(a\,x\,b) = \mathrm{PTO}_{\mathcal{E}}(a\,x\,b)$. *Both executions terminate on the same step. The complete grids and head positions agree at every step, including the final configuration. The final grid contains only the* $(m + n)$*-bit representation of* AB*, with leading-zero padding, on row* $n - 1$ *in columns* $-n, \ldots, m - 1$. *The final head stops at* $(-n - 1, n - 1)$.

The theorem concerns autonomous executions of the exported discrete controller on the stated input domain.

2 Expert Behavior

2.1 Copying and Partial Products

Write $b = b_{n-1} \cdots b_0$, so $B = \sum_{i=0}^{n-1} b_i 2^i$. Assign global weight 2^{m-1-z} to column z. Column $m-1$ carries the units digit of the final product.

Lemma 1 (Copy Staircase)**.** *Before product formation, the expert constructs exactly n occupied rows. For $0 \le i < n$, row i contains*

$$a\, x\, b_{n-1} \cdots b_i \quad \text{starting in column } -i. \tag{1}$$

The last row contains one right-operand bit. All remaining cells contain blanks.

Proof. Row 0 contains the input. On row i, the expert scans to the first blank, steps left to b_i, then steps left again. A digit at this position implies at least two remaining right-operand bits. The expert copies every symbol except b_i one row down and one column left, proceeding from right to left. Every destination contains a blank before writing; the source word remains unchanged.

The first descent reaches a blank on the new row. Each later descent reaches the copied right neighbor: a digit, except directly below the last digit of a, where the copied separator x occupies the lower cell. The cell below the source separator contains a digit. After copying the first digit of a, the expert reads the blank immediately to the left and descends to the first digit of the new row.

The new row satisfies (1) for $i+1$, with one fewer right-operand bit. When one right-operand bit remains, the second left step reads x and starts product formation. Thus $n-1$ copies produce exactly n occupied rows. □

Lemma 2 (Aligned Partial Products)**.** *After product formation, row i contains the $(m+1)$-bit representation of Ab_i in columns $-i-1, \ldots, m-i-1$, with a leading 0. In global column weights, row i represents $Ab_i 2^i$. All remaining cells contain blanks.*

Proof. The expert forms products from row $n-1$ upwards. On row i, the expert erases the rightmost factor bit b_i, the remaining factor prefix, and x, then walks left across a. For $b_i = 0$, the expert writes zeros; for $b_i = 1$, the expert preserves the digits of a. Both branches write one additional 0 on the left blank. The resulting word has the claimed support and value, with a leading zero because $Ab_i < 2^m$.

The expert scans right to the blank at column $m-i$ and moves up. For $i > 0$, the upper cell contains the last digit of a on row $i-1$; the next cell contains x. The expert scans the remaining factor prefix, selects the last bit, and repeats. Each upper prefix contains at least two bits, so another digit follows the selected bit during leftward erasure. For $i = 0$, the upper cell contains a blank, ending product formation and starting summation on row 0. □

Both passes terminate: each copy shortens the remaining factor prefix, and product formation visits each of the n rows once. Every horizontal scan crosses a finite word.

2.2 A Fixed-Width Addition Invariant

Lemma 3 (Accumulated Product)**.** *After merging through row k, $0 \le k < n$, all rows above k contain only blanks. Row k contains a contiguous $(m + k + 1)$-bit accumulator on columns $-k - 1, \ldots, m - 1$, with value*

$$S_k = A \sum_{i=0}^{k} b_i 2^i.$$

Rows below k retain the partial products from Lemma 2. At the inspection of the next row, the expert occupies state 20 with head $(-k - 2, k + 1)$.

Proof. For $k = 0$, the initial left scan crosses the product on row 0, reads the leading 0, reaches the blank at column -2, and descends.

If row $k + 1$ exists, the inspected cell contains the leading 0 at column $-k - 2$. The expert scans right across the lower product to the first blank, at column $m - k - 1$. Moving up reaches an accumulator digit because

$$-k - 1 \;\le\; m - k - 1 \;\le\; m - 1.$$

The expert scans the remaining upper digits, reaches the right blank, and steps left to column $m - 1$. The lower cell contains a blank because the original lower product ends at $m - k - 2$. The first column therefore adds the upper bit to zero without a carry.

The expert processes the remaining columns from right to left. For upper bit u, lower bit v (with a blank counting as zero), and incoming carry c, the update satisfies

$$u + v + c = w + 2c', \qquad w \in \{0, 1\}, \quad c' \in \{0, 1\}. \tag{2}$$

The expert erases the upper bit, writes w below, and stores c' in the control state. The processed lower suffix contains the correct sum suffix, the unprocessed cells retain the previous values, and the carry records the contribution to the next column. Equation (2) preserves this invariant.

The original lower word extends one column farther left than the upper word, with a leading 0 at $-k - 2$. After consuming the upper word, carry zero preserves this bit, whereas carry one changes the bit to 1 and clears. The expert reaches the blank at $-k - 3$ on row $k + 1$ and descends. Row k now contains only blanks, and row $k + 1$ represents $S_k + Ab_{k+1}2^{k+1} = S_{k+1}$ on the claimed support. □

Lemma 4 (Carry Confinement)**.** *During an upper-digit update with incoming carry 1, the lower cell contains an original binary digit. If the carry outlives the upper word, the next lower cell contains the original leading 0.*

Proof. A blank gap lies to the right of the original lower product. The first column has carry zero, and $u + 0 + 0 \le 1$, so no carry originates in the gap. A carry first arises inside the lower digit block. Further leftward steps remain inside that block until the leading zero. The upper word ends one column before that zero, where $0 + 0 + 1 = 1$ clears the carry. □

Each merge consumes finitely many upper digits and at most one extra carry column. After $n - 1$ merges, state 20 reads the blank below the final row and moves up to state 30, which emits the final S action and enters halt state 31. Lemma 3 gives the $(m + n)$-bit product and final head $(-n - 1, n - 1)$. All other cells contain blanks.

3 Observation Protocol

The observation protocol refines expert states by geometric context. Each phase records the permitted read symbols and successor phases. Section 4 gives the complete protocol together with the controller-state relation; the shared tables avoid repeating the same phase definitions.

Lemma 5 (Protocol Coverage)**.** *Every expert execution on a well-formed multiplication follows the* 94 *phases in Section 4, from phase* 0 *to phase* 93.

Proof. The copy and product invariants (Lemmas 1–2) give all copying and product-reading conditions. The scan annotations remember unchanged cells and distinguish one digit from at least two; the explanations in Sections 4.1–4.3 prove these refinements. Lemmas 3–4 and the blank-gap argument in Section 4.4 supply the addition and boundary conditions. Induction over each transition, each copied row and each merged row establishes coverage. The finite-pass arguments in the expert invariants give termination in phase 93. □

4 Simulation Certificate

4.1 Copying and Remembered Factor Bits

The next four tables define a finite protocol and simulation relation. Phase i pairs expert state g_i with a controller state $q \in Q_i$, with identical grids and heads. An entry $x : (j, w, M)$ means read x, write w, move M, and advance to *phase* j. The symbol $d \in \{0, 1\}$ means either digit, consistently within that entry. For *each* $q \in Q_i$, the actual successor lies in Q_j.

i	Phase	g_i	Q_i	Read : (next phase, write, move)
0	First left-operand digit	0	$\{0\}$	$d : (1, d, \mathsf{R})$
1	Scan left operand	0	$\{0, 24\}$	$d : (1, d, \mathsf{R})$ $x : (2, x, \mathsf{R})$
2	First right-operand digit	0	$\{7\}$	$0 : (3, 0, \mathsf{R})$ $1 : (4, 1, \mathsf{R})$
3	Scan factor; last bit 0	0	$\{48\}$	$0 : (3, 0, \mathsf{R})$ $1 : (4, 1, \mathsf{R})$ $\lambda : (5, \lambda, \mathsf{L})$
4	Scan factor; last bit 1	0	$\{48\}$	$0 : (3, 0, \mathsf{R})$ $1 : (4, 1, \mathsf{R})$ $\lambda : (6, \lambda, \mathsf{L})$
5	Revisit last factor bit 0	1	$\{14\}$	$0 : (7, 0, \mathsf{L})$
6	Revisit last factor bit 1	1	$\{14\}$	$1 : (8, 1, \mathsf{L})$
7	Copy or use remembered bit 0	2	$\{49\}$	$0 : (9, 0, \mathsf{D})$ $1 : (11, 1, \mathsf{D})$ $x : (25, x, \mathsf{R})$
8	Copy or use remembered bit 1	2	$\{19\}$	$0 : (9, 0, \mathsf{D})$ $1 : (11, 1, \mathsf{D})$ $x : (26, x, \mathsf{R})$
9	First copied 0: descend	3	$\{16\}$	$\lambda : (10, \lambda, \mathsf{L})$
10	First copied 0: write	6	$\{13\}$	$\lambda : (13, 0, \mathsf{U})$
11	First copied 1: descend	4	$\{4\}$	$\lambda : (12, \lambda, \mathsf{L})$
12	First copied 1: write	7	$\{41\}$	$\lambda : (13, 1, \mathsf{U})$
13	Continue after first copy	9	$\{35\}$	$0 : (14, 0, \mathsf{D})$ $1 : (16, 1, \mathsf{D})$ $x : (19, x, \mathsf{D})$
14	Copy 0 above a lower digit	3	$\{44, 47\}$	$d : (15, d, \mathsf{L})$
15	Write copied 0	6	$\{9, 49\}$	$\lambda : (18, 0, \mathsf{U})$
16	Copy 1 above a lower digit	4	$\{26\}$	$d : (17, d, \mathsf{L})$
17	Write copied 1	7	$\{2, 3\}$	$\lambda : (18, 1, \mathsf{U})$
18	Continue the copy	9	$\{34\}$	$0 : (14, 0, \mathsf{D})$ $1 : (16, 1, \mathsf{D})$ $x : (19, x, \mathsf{D})$ $\lambda : (24, \lambda, \mathsf{D})$
19	Descend below operator	5	$\{45\}$	$d : (20, d, \mathsf{L})$
20	Write shifted operator	8	$\{5\}$	$\lambda : (21, x, \mathsf{U})$
21	Copy last left-operand digit	9	$\{34\}$	$0 : (22, 0, \mathsf{D})$ $1 : (23, 1, \mathsf{D})$
22	Copied 0 above lower operator	3	$\{44\}$	$x : (15, x, \mathsf{L})$
23	Copied 1 above lower operator	4	$\{26\}$	$x : (17, x, \mathsf{L})$
24	Begin scan on copied row	0	$\{36\}$	$d : (1, d, \mathsf{R})$

Coverage. Both operands contain at least one digit. The initial scan therefore meets a digit, x, and a right-operand digit before a blank. Phases 3–8 remember the last factor bit: the scan's last digit, the digit revisited by the first left step, and (if only one factor bit remains) the digit selected for multiplication all refer to the same unchanged cell.

Lemma 1 proves the remaining restrictions: the first descent sees a blank; subsequent descents see the already copied right-neighbor digit, except below the last left-operand digit, where the lower cell contains x. Every destination contains a blank. Copying x exposes a source digit. Restarting on a copied row also reads a digit. These conditions hold for arbitrary operand lengths and values.

4.2 Forming and Scanning Partial Products

The product scan starts just to the right of the new padding zero. “One” denotes a scan after exactly one product digit; the remaining scan phases follow at least two product digits. Each scan also records the most recently read digit.

i	Phase	g_i	Q_i	Read : (next phase, write, move)		
25	Select remembered factor bit 0	10	$\{10\}$	$0 : (27, \lambda, \mathsf{L})$		
26	Select remembered factor bit 1	10	$\{3\}$	$1 : (28, \lambda, \mathsf{L})$		
27	Erase operator for factor 0	11	$\{10\}$	$x : (29, \lambda, \mathsf{L})$		
28	Erase operator for factor 1	12	$\{4\}$	$x : (31, \lambda, \mathsf{L})$		
29	First digit of product by 0	13	$\{25\}$	$d : (30, 0, \mathsf{L})$		
30	Continue product by 0	13	$\{25\}$	$d : (30, 0, \mathsf{L})$	$\lambda : (33, 0, \mathsf{R})$	
31	First digit of product by 1	14	$\{6\}$	$d : (32, d, \mathsf{L})$		
32	Continue product by 1	14	$\{6\}$	$d : (32, d, \mathsf{L})$	$\lambda : (33, 0, \mathsf{R})$	
33	First digit after product padding	15	$\{15, 36\}$	$0 : (34, 0, \mathsf{R})$	$1 : (35, 1, \mathsf{R})$	
34	After one product digit 0	15	$\{0\}$	$0 : (36, 0, \mathsf{R})$	$1 : (37, 1, \mathsf{R})$	$\lambda : (38, \lambda, \mathsf{U})$
35	After one product digit 1	15	$\{24\}$	$0 : (36, 0, \mathsf{R})$	$1 : (37, 1, \mathsf{R})$	$\lambda : (39, \lambda, \mathsf{U})$
36	Product scan; last digit 0	15	$\{24\}$	$0 : (36, 0, \mathsf{R})$	$1 : (37, 1, \mathsf{R})$	$\lambda : (40, \lambda, \mathsf{U})$
37	Product scan; last digit 1	15	$\{24\}$	$0 : (36, 0, \mathsf{R})$	$1 : (37, 1, \mathsf{R})$	$\lambda : (41, \lambda, \mathsf{U})$
38	Inspect above one-digit product 0	16	$\{39\}$	$d : (42, d, \mathsf{R})$	$\lambda : (52, \lambda, \mathsf{D})$	
39	Inspect above one-digit product 1	16	$\{17\}$	$d : (42, d, \mathsf{R})$	$\lambda : (53, \lambda, \mathsf{D})$	
40	Inspect above; last digit 0	16	$\{17\}$	$d : (42, d, \mathsf{R})$	$\lambda : (54, \lambda, \mathsf{D})$	
41	Inspect above; last digit 1	16	$\{17\}$	$d : (42, d, \mathsf{R})$	$\lambda : (55, \lambda, \mathsf{D})$	
42	Cross upper operator	17	$\{8, 43\}$	$x : (43, x, \mathsf{R})$		
43	First bit in upper factor prefix	17	$\{16\}$	$0 : (44, 0, \mathsf{R})$	$1 : (45, 1, \mathsf{R})$	
44	Scan factor; last bit 0	17	$\{16\}$	$0 : (44, 0, \mathsf{R})$	$1 : (45, 1, \mathsf{R})$	$\lambda : (46, \lambda, \mathsf{L})$
45	Scan factor; last bit 1	17	$\{16\}$	$0 : (44, 0, \mathsf{R})$	$1 : (45, 1, \mathsf{R})$	$\lambda : (47, \lambda, \mathsf{L})$
46	Select remembered factor bit 0	10	$\{13\}$	$0 : (48, \lambda, \mathsf{L})$		
47	Select remembered factor bit 1	10	$\{13\}$	$1 : (50, \lambda, \mathsf{L})$		
48	First remaining bit to erase (0)	11	$\{27\}$	$d : (49, \lambda, \mathsf{L})$		
49	Erase factor prefix (0)	11	$\{10\}$	$d : (49, \lambda, \mathsf{L})$	$x : (29, \lambda, \mathsf{L})$	
50	First remaining bit to erase (1)	12	$\{22\}$	$d : (51, \lambda, \mathsf{L})$		
51	Erase factor prefix (1)	12	$\{1\}$	$d : (51, \lambda, \mathsf{L})$	$x : (31, \lambda, \mathsf{L})$	

Coverage. Lemma 2 gives a nonempty block of m product digits after the padding zero. This justifies the first-digit phase and the split into exactly one or at least two scanned digits. At the right blank, the most recently read digit equals the units digit.

The bottom row contains one factor bit; erasing that bit exposes x. Every higher row contains at least two factor bits, so another digit precedes x during erasure. An upward return from a completed row reaches either a blank or the last left-operand digit followed by x. The upper factor scan records the last bit, which remains unchanged until selection. All these facts follow from the copy staircase.

4.3 Starting the Sum and Aligning a Row

These phases retain the last product digit until the next visit and distinguish a one-digit left operand from a longer operand. The remembered bit and length class serve as proof annotations without adding machine registers.

i	Phase	g_i	Q_i	Read : (next phase, write, move)		
52	Begin one-digit sum; last bit 0	18	$\{20\}$	$\lambda : (56, \lambda, \mathsf{L})$		
53	Begin one-digit sum; last bit 1	18	$\{42\}$	$\lambda : (57, \lambda, \mathsf{L})$		
54	Begin sum; remember last digit 0	18	$\{42\}$	$\lambda : (58, \lambda, \mathsf{L})$		
55	Begin sum; remember last digit 1	18	$\{42\}$	$\lambda : (59, \lambda, \mathsf{L})$		
56	Revisit sole product digit 0	19	$\{28\}$	$0 : (60, 0, \mathsf{L})$		
57	Revisit sole product digit 1	19	$\{11\}$	$1 : (60, 1, \mathsf{L})$		
58	Revisit product units digit 0	19	$\{11\}$	$0 : (62, 0, \mathsf{L})$		
59	Revisit product units digit 1	19	$\{11\}$	$1 : (62, 1, \mathsf{L})$		
60	Read the sole leading zero	19	$\{2\}$	$0 : (61, 0, \mathsf{L})$		
61	Left boundary after sole digit	19	$\{23\}$	$\lambda : (66, \lambda, \mathsf{D})$		
62	Second digit of initial left scan	19	$\{2, 26\}$	$d : (63, d, \mathsf{L})$		
63	Third digit of initial left scan	19	$\{2, 23\}$	$0 : (64, 0, \mathsf{L})$	$1 : (65, 1, \mathsf{L})$	
64	Left scan after reading zero	19	$\{23\}$	$0 : (64, 0, \mathsf{L})$	$1 : (65, 1, \mathsf{L})$	$\lambda : (66, \lambda, \mathsf{D})$
65	Left scan after reading one	19	$\{23\}$	$0 : (64, 0, \mathsf{L})$	$1 : (65, 1, \mathsf{L})$	
66	Inspect fresh lower leading cell	20	$\{12, 36\}$	$0 : (67, 0, \mathsf{R})$	$\lambda : (92, \lambda, \mathsf{U})$	
67	First digit after lower leading zero	21	$\{0\}$	$d : (68, d, \mathsf{R})$		
68	Scan the lower partial product	21	$\{24\}$	$d : (68, d, \mathsf{R})$	$\lambda : (69, \lambda, \mathsf{U})$	
69	First upper alignment digit	28	$\{17\}$	$d : (70, d, \mathsf{R})$		
70	Next upper alignment cell	28	$\{8\}$	$d : (71, d, \mathsf{R})$	$\lambda : (72, \lambda, \mathsf{L})$	
71	Finish aligning the upper row	28	$\{4, 8\}$	$d : (71, d, \mathsf{R})$	$\lambda : (72, \lambda, \mathsf{L})$	

Initial Left-Scan Coverage. On the top row, the product's units digit has not changed between the right scan and the first left step of summation. If $m = 1$, the full padded word equals $0d$: scanning left reads d, then 0, then a blank (phases 56–57, 60–61). If $m \geq 2$, the padded word has at least three digits, so two more digits must follow the first one (phases 62–63). In both cases the padding zero immediately precedes the left blank.

Alignment Coverage. Lemma 3 gives either a blank in the inspected lower cell (no remaining row) or the next row's leading 0. After that zero comes at least one digit. The upward move from the lower right boundary reaches an upper digit, so alignment begins inside the upper word. The expert scans to the right blank and then returns left to the units digit. The first lower units cell contains a blank. These facts justify every alignment branch here and the first three phases in the next certificate subsection.

4.4 Column Addition, Carries and Stopping

Separate phases track carry-zero returns from lower blanks and lower digits. This distinction prevents an impossible early end of the upper word from entering the proof protocol.

i	Phase	g_i	Q_i	Read : (next phase, write, move)		
72	Read upper units digit	22	$\{32, 41\}$	$0 : (73, \lambda, \mathsf{D})$	$1 : (74, \lambda, \mathsf{D})$	
73	Write units digit 0 below	24	$\{27, 46\}$	$\lambda : (88, 0, \mathsf{U})$		
74	Write units digit 1 below	25	$\{22\}$	$\lambda : (88, 1, \mathsf{U})$		
75	Return above, carry zero	23	$\{50\}$	$\lambda : (76, \lambda, \mathsf{L})$		
76	Read upper bit, carry zero	22	$\{37\}$	$0 : (77, \lambda, \mathsf{D})$	$1 : (78, \lambda, \mathsf{D})$	$\lambda : (83, \lambda, \mathsf{D})$
77	Add value zero below	24	$\{30\}$	$d : (75, d, \mathsf{U})$	$\lambda : (90, 0, \mathsf{U})$	
78	Add value one below	25	$\{29\}$	$\lambda : (90, 1, \mathsf{U})$	$0 : (75, 1, \mathsf{U})$	$1 : (79, 0, \mathsf{U})$
79	Return above, carry one	27	$\{10\}$	$\lambda : (80, \lambda, \mathsf{L})$		
80	Read upper bit, carry one	26	$\{38\}$	$0 : (81, \lambda, \mathsf{D})$	$1 : (82, \lambda, \mathsf{D})$	$\lambda : (85, \lambda, \mathsf{D})$
81	Add one inside lower digit block	25	$\{29\}$	$0 : (75, 1, \mathsf{U})$	$1 : (79, 0, \mathsf{U})$	
82	Add two inside lower digit block	29	$\{9\}$	$d : (79, d, \mathsf{U})$		
83	Preserve unneeded leading zero	19	$\{20\}$	$0 : (84, 0, \mathsf{L})$		
84	Blank to the left of merged row	19	$\{31, 34\}$	$\lambda : (66, \lambda, \mathsf{D})$		
85	Apply final carry to leading zero	25	$\{42\}$	$0 : (86, 1, \mathsf{U})$		
86	Return after final carry	23	$\{47\}$	$\lambda : (87, \lambda, \mathsf{L})$		
87	Step down left of merged row	22	$\{33\}$	$\lambda : (84, \lambda, \mathsf{D})$		
88	Return after units digit	23	$\{18, 21\}$	$\lambda : (89, \lambda, \mathsf{L})$		
89	Read the second upper digit	22	$\{21\}$	$0 : (77, \lambda, \mathsf{D})$	$1 : (78, \lambda, \mathsf{D})$	
90	Return from the lower blank gap	23	$\{21\}$	$\lambda : (91, \lambda, \mathsf{L})$		
91	Upper digit after lower blank gap	22	$\{21\}$	$0 : (77, \lambda, \mathsf{D})$	$1 : (78, \lambda, \mathsf{D})$	
92	Emit final stop	30	$\{35, 40\}$	$\lambda : (93, \lambda, \mathsf{S})$		
93	Halted	31	$\{51\}$	–		

The Lower Blank Gap. While merging row k into row $k + 1$, the upper word begins at $-k - 1$; the original lower word ends at $m - k - 2 \geq -k - 1$. A blank below an upper digit must therefore lie to the right of that lower endpoint. The next column left still has index at least $-k - 1$, so the next upper reading contains a digit. This proves the restriction in phase 91. Independently, the upper word has length $m + k + 1 \geq 2$, proving the second-digit restriction in phase 89.

Carry and Boundary Coverage. Lemma 4 excludes lower blanks in the carry-one digit phases and forces 0 for the final carry extension. Returning above a written lower digit reads the upper blank just erased. When the upper word ends, the next lower cell contains the original leading zero: carry zero preserves this bit, whereas carry one changes the bit to 1. Both branches reach the blank to the left of the merged row. When no next row remains, the upward move reaches a blank, and the expert emits the final S. Phase 93 has no further PTO actions.

Local Transition Identity. For every listed input branch $x : (j, w, M)$ and every $q \in Q_i$, direct comparison with the two exported tables gives

$$\delta_{\mathcal{E}}(g_i, x) = (g_j, w, M), \qquad \delta_C(q, x) = (q', w, M) \quad \text{for some } q' \in Q_j. \tag{3}$$

The certificate contains 109 phase–state pairs and 201 local checks, covering 133 distinct nonterminal controller entries. A transition reaches a halt state exactly when $j = 93$, with $M = S$. Initially $g_0 = 0$ and $Q_0 = \{0\}$. Protocol coverage excludes the remaining controller entries on valid inputs.

5 Equivalence Proof

Proof of Theorem 1. Initially both configurations have the same grid and head, with control states 0 related by phase 0. Suppose the relation holds before a controller step. Lemma 5 supplies an allowed read symbol x. Equal grids and heads give the same read symbol to both controllers. Equation (3) gives equal writes and motions and preserves the relation between successor states. The next grids and heads therefore agree.

Induction establishes equality of every finite execution prefix. The terminal condition permits a halt exactly on an edge to the terminal phase. The finite copy, product, and merge passes establish termination. Both controllers therefore halt together with equal complete PTOs and final configurations. Lemma 3 gives the product, padding, and final head. □

6 Verification and Artifacts

Verification. Run `python3 verify_equivalence.py --quick` from this proof directory to check file fingerprints, CSV/JSON agreement, the exported expert against the `mult_grid(2)` source, and all 201 certificate obligations. Omitting `--quick` additionally compares independent executions on all 15,876 operand pairs of lengths 1–6, plus 45 directed and 128 fixed-seed random cases, including 1024-bit operands. All 10,184,741 PTO steps agree. The checks also compare final grids, heads, and padded products. These tests exercise 132 distinct nonterminal entries. The unbounded result follows from the invariants and induction above.

Artifact Identity. The title page gives the repository-relative case directory. This proof directory contains the controller and expert exports, reference code, protocol, checker, and saved verification report. The verification script stores or reads SHA-256 checksums for these artifacts. A checksum identifies the exact bytes of a file; the checker compares the stored and computed values to detect changes. These checks bind the certificate to the supplied exports. The invariants, protocol coverage, and local transition identity establish equivalence.